

Test-Time Computation & Reinforcement Learning

Yunfan Zhang

Columbia COMS 4705 NLP

Spring 2026

Structure of This Lecture

- **Why we need Test-Time Computation and Reinforcement Learning?**
 - CoT & ReAct Recap
 - Why we need TTC and RL
- **RL Overview**
 - The Bitter Lesson; RL is search and learning
 - Process Reward vs Outcome / Verifiable Reward
- **RL Algorithm Deep-dive**
 - The GRPO Algorithm
 - Advantage, Clipping, KL Penalty
 - PPO vs GRPO
 - Practical Guides and Observations
- **Conclusions**
 - Ethics and Societal Impacts

Test-Time Computation is important for reasoning tasks (CoT)

Please use each of the following numbers to derive 0.3: 1, 5, 6, 8, 9.
You are only allowed to use addition, subtractions, multiplications, divisions, and parenthesis. You must use all numbers exactly once.



⊗ Thought for 268 seconds ▾

- We need to use each of the numbers 1, 5, 6, 8, 9 exactly once to get 0.3. Allowed operations: addition, subtraction, multiplication, division, and parentheses. We must use all numbers exactly once.
- We want to get 0.3, which is $\frac{3}{10}$. So we need to combine these numbers to form a fraction equal to $\frac{3}{10}$.
- Let's list the numbers: 1,5,6,8,9. We need to use all five.
- We can think of combining them into an expression that equals 0.3. Perhaps something like $(a-b)/(c*d)$ or something.
- We need $0.3 = \frac{3}{10}$. So maybe we can make numerator 3 and denominator 10 using these numbers.
- How to make 3? Possible: $9-6=3$, or $8-5=3$, or $5-1=4$? but we have to use all numbers.
- Denominator 10: $5+5=10$ but we only have one 5. Maybe $8+1=9$? but we need to use all.
- We have to use all numbers exactly once. So we need to combine them into an expression that simplifies to $\frac{3}{10}$.
- Let's try to make numerator = 3 and denominator = 10.
- Numerator possibilities: $9-6=3$, or $8-5=3$, or $(9-6)$ etc.

Using the numbers 1, 5, 6, 8, 9 each exactly once, we can derive 0.3 with the following expression:

$$\frac{5 + 6 - 8}{9 + 1} = \frac{3}{10} = 0.3$$

This uses addition, subtraction, division, and parentheses as allowed.

With Test-Time Compute / Chain of Thought

Please use each of the following numbers to derive 0.3: 1, 5, 6, 8, 9.
You are only allowed to use addition, subtractions, multiplications, divisions, and parenthesis. You must use all numbers exactly once.
Do not think step by step. Answer directly with an equation.



$$(9 - 8) \times (6 \div (5 \times 1))$$

Without Test-Time Compute / Chain of Thought

Test-Time Compute is essential for agentic tasks (ReAct)

yunfan ✨ Please check the verl implementation and see if it implements sequence packing correctly for RL + FSDP. See if the attention mechanism correctly handles packed samples so that the model never attends across sample boundaries.

• *Let me understand the task:*

1. *Check the verl implementation for sequence packing*
2. *Specifically for RL + FSDP*
3. *Verify that the attention mechanism correctly handles packed samples*
4. *Ensure the model never attends across sample boundaries*



This is a complex task that requires understanding:

- *The verl codebase structure*
- *How sequence packing is implemented*
- *How FSDP is used in the RL context*
- *How attention masks are handled for packed sequences*

Let me start by exploring the verl directory to understand the structure and find relevant files related to sequence packing and attention mechanisms.

- *I'll check the verl implementation to see if sequence packing is correctly implemented for RL + FSDP, particularly focusing on attention mechanisms and boundary handling.*
- *Used Bash (find verl -name "*.py" | head -20)*
- *I need to find the core implementation files related to sequence packing and attention mechanisms in the verl library. Let me search for files that might contain sequence packing logic.*
- *Used Grep (sequence.*pack|pack.*sequ...on.*mask|flash.*attention)*
- *Let me look for specific files related to attention mechanisms and sequence packing. The key files to check would be:*



1. [verl/utils/attention_utils.py](#) - *likely contains attention utilities*
2. [verl/workers/fsdp_workers.py](#) - *FSDP workers implementation*
3. [verl/workers/actor/dp_actor.py](#) - *actor implementation*
4. *Files related to models and transformers*

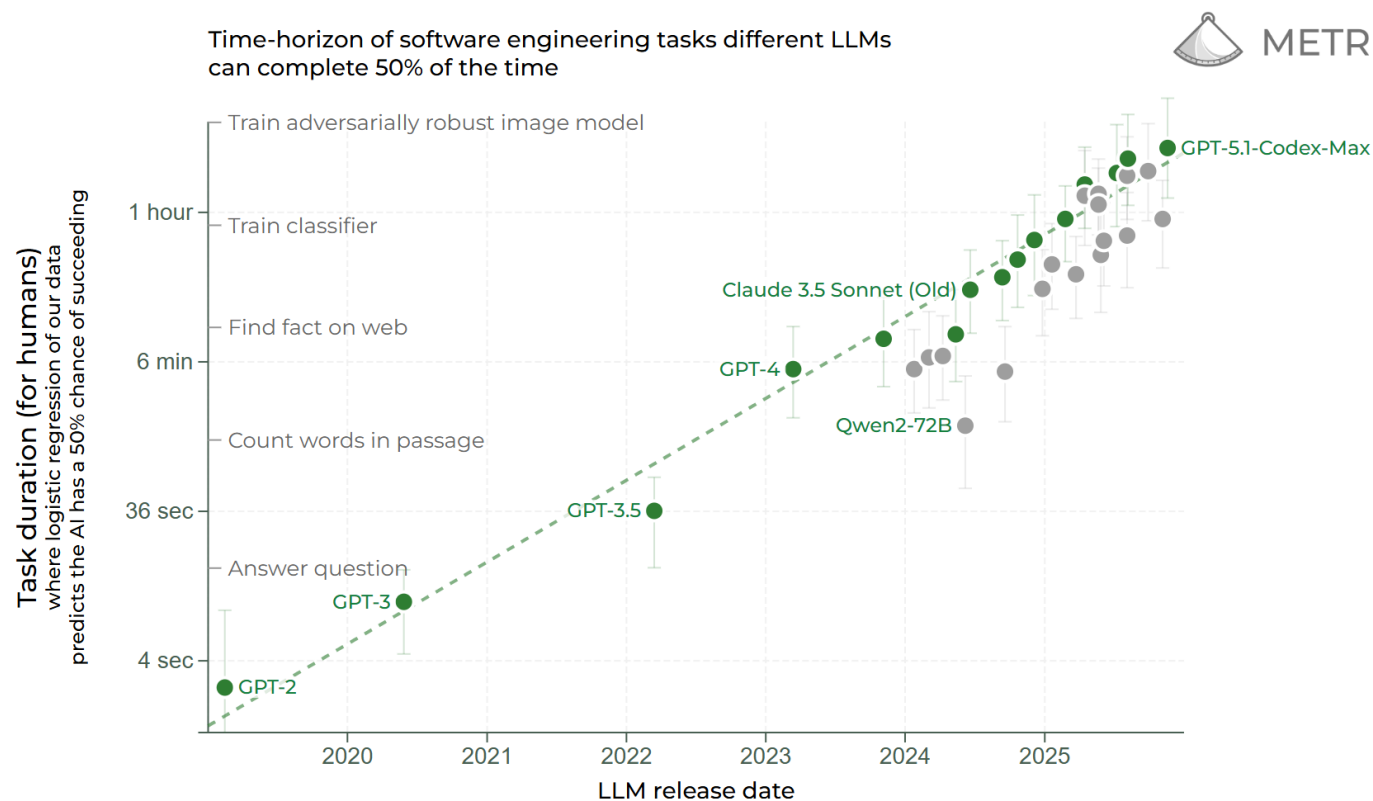


Let me start by examining these files.

- *Used ReadFile (verl/verl/utils/attention_utils.py)*
- *Invalid path*
- *I need to provide the absolute path. Let me fix that*

Why do we need Test-Time Computation?

We need test-time compute because complex, long horizon tasks are never completed in a single step.



Why do we need Reinforcement Learning?



Why do we need Reinforcement Learning?

- SFT:
 - Memorize training data.
 - Learning how others might solve it. Not how you would solve it.
- Reinforcement Learning:
 - **On-policy updates:** learn from your own mistakes through search and learning
 - Goal driven, mitigate exponentially increasing errors
 - Solve the problems that the humans do not know how to solve

The Bitter Lesson – Rich Sutton

“The biggest lesson that can be read from 70 years of AI research is that **general methods that leverage computation are ultimately the most effective**, and by a large margin.”

“One thing that should be learned from the bitter lesson is the great power of general purpose methods, of methods that continue to scale with increased computation even as the available computation becomes very great. The two methods that seem to scale arbitrarily in this way are **search and learning**.”

Reinforcement Learning is Search and Learning

- **Search:** For every sample in the training dataset, generate multiple attempts / rollouts from a policy model (e.g. the LLM you are training).
- **Reward:** assign score(s) to each attempt, based on how good it is.
- **Learning:** update the policy model based on the reward(s).
 - If a rollout is good (high reward), increase $P(\text{rollout} \mid \text{sample input})$
 - If a rollout is bad (low reward), decrease $P(\text{rollout} \mid \text{sample input})$
 - A.k.a: make good rollouts more likely to occur again, bad rollouts less likely to occur again.

Process Reward vs Outcome / Verifiable Reward

- **Process Rewards:** assign rewards for each of the intermediate steps, as well as the final answer.
 - E.g. use another LLM to grade a proposed solution step by step.
 - RL with Human Feedback is almost always PRM
- **Outcome Rewards / RL with Verifiable Rewards (RLVR):** assign rewards based on verifiable outcomes. E.g.:
 - For question answering, does this attempt provide the correct answer?
 - For coding problems, the percentage of test cases does this attempt pass?

Issues with Process Rewards

- **Reward hacking:** the policy model attains a high reward w.r.t the Process Reward Model (PRM), but the performance on the task deteriorate.
 - A.k.a **Cheating**
 - Learned to perform **adversarial attack on the PRM**
- Given enough rollouts (attempts), the policy model will explore every possible strategy to get a high reward:
 - No matter how **illegal** it may seem
 - No matter how impossible it is
- Also, where does the data to train PRM come from?
 - Human expert annotation is not scalable

RLVR is a General Paradigm

- Works on any task where the **proposed solution can be verified**, and the model as a **non-zero chance of solving** some samples in the training set.
 - Mathematics
 - Programming
 - Agents
 - Question Answering
 - Science & Engineering?
- RLVR does not necessarily require a ground truth solution
- No need to design human inspired heuristics / algorithms.
- No need for step-by-step annotations from human experts.
- RLVR is much more resistant to reward hacking.
 - But not reward-hacking free if your verifier / environment is hackable!

The GRPO Algorithm

- For every batch D_b in dataset D :
 - Sample a group of rollouts D_r from policy model π_θ
 - Assign rewards R to the rollouts D_r
 - For every mini batch D_{mini} in D_r :
 - Update the policy model π_θ according to the GRPO Objective

GRPO Objective

$$\mathcal{J}_{GRPO}(\theta) = \mathbb{E}[q \sim P(Q), \{o_i\}_{i=1}^G \sim \pi_{\theta_{old}}(O|q)]$$

$$\frac{1}{G} \sum_{i=1}^G \frac{1}{|o_i|} \sum_{t=1}^{|o_i|} \left\{ \min \left[\frac{\pi_{\theta}(o_{i,t}|q, o_{i,<t})}{\pi_{\theta_{old}}(o_{i,t}|q, o_{i,<t})} \hat{A}_{i,t}, \text{clip} \left(\frac{\pi_{\theta}(o_{i,t}|q, o_{i,<t})}{\pi_{\theta_{old}}(o_{i,t}|q, o_{i,<t})}, 1 - \varepsilon, 1 + \varepsilon \right) \hat{A}_{i,t} \right] - \beta \mathbb{D}_{KL} [\pi_{\theta} || \pi_{ref}] \right\},$$

$$A_i = \frac{r_i - \text{mean}(\{r_1, r_2, \dots, r_G\})}{\text{std}(\{r_1, r_2, \dots, r_G\})}.$$

$$\mathbb{D}_{KL} [\pi_{\theta} || \pi_{ref}] = \frac{\pi_{ref}(o_{i,t}|q, o_{i,<t})}{\pi_{\theta}(o_{i,t}|q, o_{i,<t})} - \log \frac{\pi_{ref}(o_{i,t}|q, o_{i,<t})}{\pi_{\theta}(o_{i,t}|q, o_{i,<t})} - 1,$$

Advantage in GRPO

$$A_i = \frac{r_i - \text{mean}(\{r_1, r_2, \dots, r_G\})}{\text{std}(\{r_1, r_2, \dots, r_G\})}.$$

- Advantage is assigned on at the action (token) level.
 - It means “how good is each action (token)?”
 - In contrast, reward is assigned at sequence level.
- All tokens in the same rollout share the same advantage A_i .
- GRPO assigns advantage by comparing the performance of each rollout against others in the group.
 - Think of it as SAT scores: your score is normalized against your peer group, who takes the same exam.
- Positive if better than the group average, negative if worse than group average

Main GRPO Update Rule

$$\frac{\pi_{\theta}(o_{i,t}|q, o_{i,<t})}{\pi_{\theta_{old}}(o_{i,t}|q, o_{i,<t})} \hat{A}_{i,t},$$

- The more a rollout **outperforms** the peer group, the **more likely** it should appear the next time.
- The more a rollout **underperforms** the peer group, the **less likely** it should appear the next time.
- If the rollout performs the same as the average of the peer group, zero gradient, and do nothing.
 - GRPO effectively skips samples where all rollouts have the same reward (e.g. all are correct; all are incorrect) -> **curriculum learning**.

Clipping

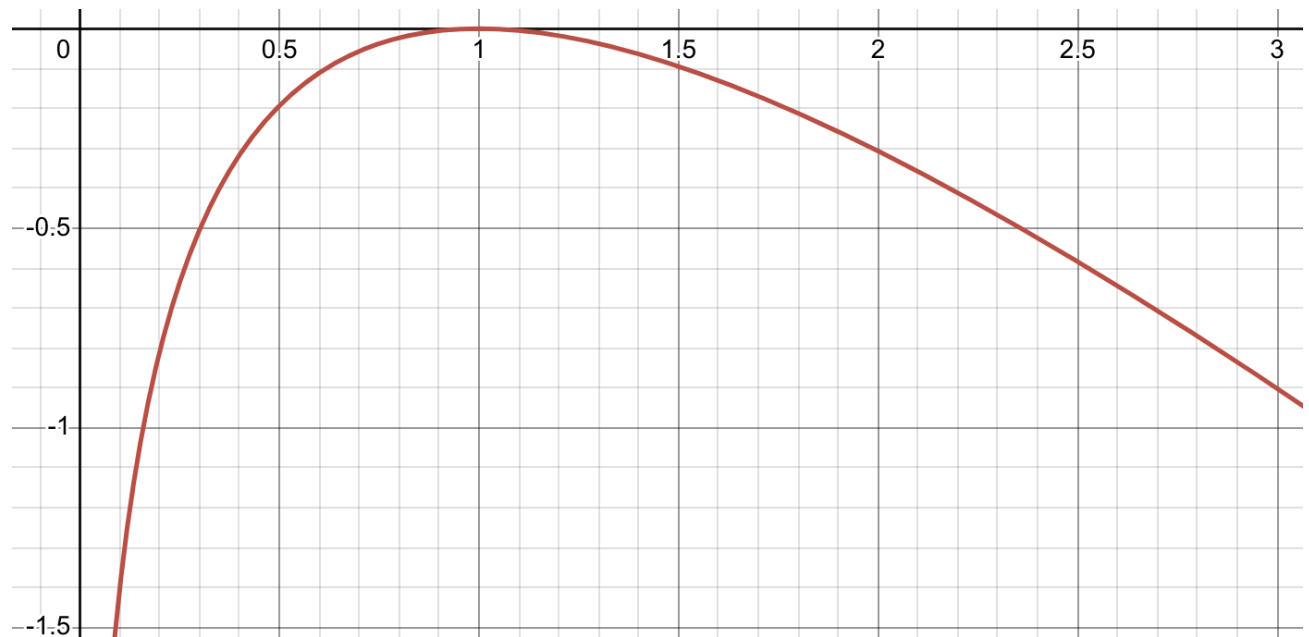
$$\min \left[\frac{\pi_{\theta}(o_{i,t}|q, o_{i,<t})}{\pi_{\theta_{old}}(o_{i,t}|q, o_{i,<t})} \hat{A}_{i,t}, \text{clip} \left(\frac{\pi_{\theta}(o_{i,t}|q, o_{i,<t})}{\pi_{\theta_{old}}(o_{i,t}|q, o_{i,<t})}, 1 - \epsilon, 1 + \epsilon \right) \hat{A}_{i,t} \right]$$

- Only kicks in when:
 - $r_i > 1 + \epsilon, A_i > 0$
 - $r_i < 1 - \epsilon, A_i < 0$
- Only kicks on or after the second mini batch
 - This is why you should not set batch size the same as mini batch
- Idea: establish a trust region to improve training stability
 - If the ratio is already too high, do not make it higher
 - If the ratio is too low, do not make it lower
 - Aka do not update too much on a batch!
- Low probability event (0.1% of the tokens) in practice

KL Penalty

$$-\beta \mathbb{D}_{KL} [\pi_{\theta} || \pi_{ref}] \quad \mathbb{D}_{KL} [\pi_{\theta} || \pi_{ref}] = \frac{\pi_{ref}(o_{i,t} | q, o_{i,<t})}{\pi_{\theta}(o_{i,t} | q, o_{i,<t})} - \log \frac{\pi_{ref}(o_{i,t} | q, o_{i,<t})}{\pi_{\theta}(o_{i,t} | q, o_{i,<t})} - 1,$$

- Reference model is the original model before any RL training.
- Regularization: the sequence probability of the current policy model should not drift too far from the original model
- This is again to improve training stability.



GRPO Objective: Putting it back together

$$\mathcal{J}_{GRPO}(\theta) = \mathbb{E}[q \sim P(Q), \{o_i\}_{i=1}^G \sim \pi_{\theta_{old}}(O|q)]$$

$$\frac{1}{G} \sum_{i=1}^G \frac{1}{|o_i|} \sum_{t=1}^{|o_i|} \left\{ \min \left[\frac{\pi_{\theta}(o_{i,t}|q, o_{i,<t})}{\pi_{\theta_{old}}(o_{i,t}|q, o_{i,<t})} \hat{A}_{i,t}, \text{clip} \left(\frac{\pi_{\theta}(o_{i,t}|q, o_{i,<t})}{\pi_{\theta_{old}}(o_{i,t}|q, o_{i,<t})}, 1 - \varepsilon, 1 + \varepsilon \right) \hat{A}_{i,t} \right] - \beta \mathbb{D}_{KL} [\pi_{\theta} || \pi_{ref}] \right\},$$

$$A_i = \frac{r_i - \text{mean}(\{r_1, r_2, \dots, r_G\})}{\text{std}(\{r_1, r_2, \dots, r_G\})}.$$

$$\mathbb{D}_{KL} [\pi_{\theta} || \pi_{ref}] = \frac{\pi_{ref}(o_{i,t}|q, o_{i,<t})}{\pi_{\theta}(o_{i,t}|q, o_{i,<t})} - \log \frac{\pi_{ref}(o_{i,t}|q, o_{i,<t})}{\pi_{\theta}(o_{i,t}|q, o_{i,<t})} - 1,$$

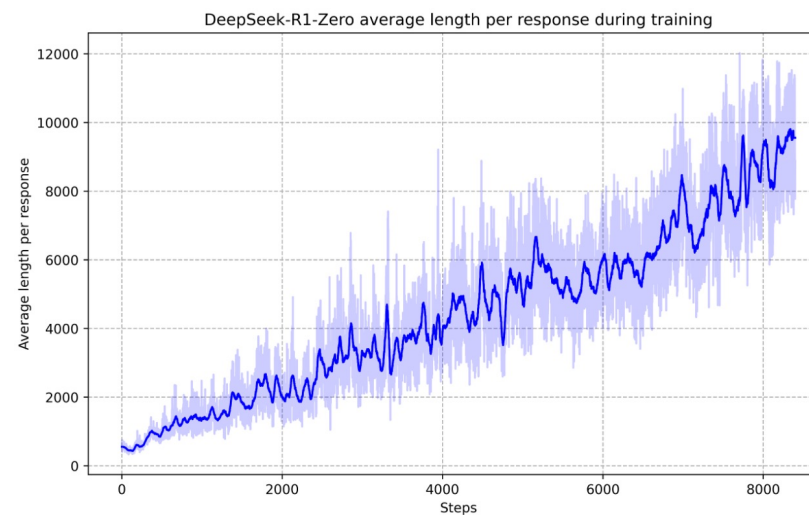
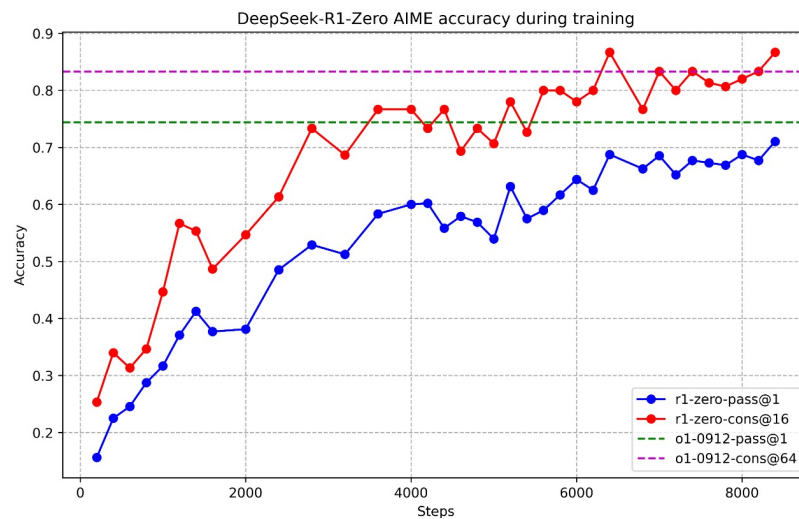
Proximal Policy Optimization (PPO)

- The major difference is how advantage $A_{i,t}$ is defined.
- Estimate: $\hat{A}_t^{\text{GAE}(\gamma,\lambda)} = \sum_{l=0}^{T-t-1} (\gamma\lambda)^l \delta_{t+l}, \quad \delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$
- V is a value model fitted by: $L^{\text{VF}}(\phi) = \mathbb{E}[(V_\phi(s_t) - \hat{V}_t)^2]$
- When $\lambda = 1, \gamma = 1$, GAE simplifies to $\hat{A}_t = G_t - V(s_t)$
- Advantage will be different token by token as depends on $V(s_t)$
 - If your value function is good, this is better than GRPO.
 - But in LLMs, there are thousands of actions, and it is hard to assign appropriate value to each of the action.
 - PPO does not guarantee curriculum learning.

Remember the following distinctions

- Process Rewards vs Outcome Rewards
- Rewards vs Advantage
- Rollout-level advantage (GRPO) vs action-level advantage (PPO)

DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning (DeepSeek-AI et al., 2025)



Question: If $a > 1$, then the sum of the real solutions of $\sqrt{a - \sqrt{a+x}} = x$ is equal to

Response: <think>

To solve the equation $\sqrt{a - \sqrt{a+x}} = x$, let's start by squaring both ...

$$\left(\sqrt{a - \sqrt{a+x}}\right)^2 = x^2 \implies a - \sqrt{a+x} = x^2.$$

Rearrange to isolate the inner square root term:

$$(a - x^2)^2 = a + x \implies a^2 - 2ax^2 + (x^2)^2 = a + x \implies x^4 - 2ax^2 - x + (a^2 - a) = 0$$

...

Wait, wait. Wait. That's an aha moment I can flag here.

Let's reevaluate this step-by-step to identify if the correct sum can be ...

We started with the equation:

$$\sqrt{a - \sqrt{a+x}} = x$$

First, let's square both sides:

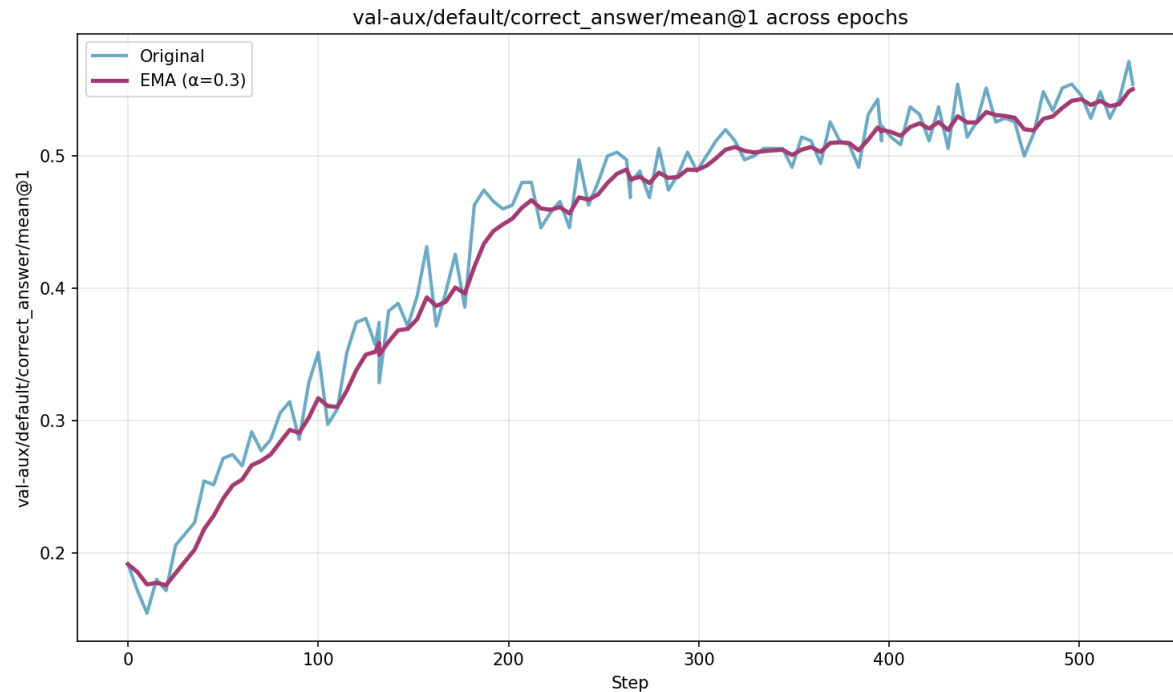
$$a - \sqrt{a+x} = x^2 \implies \sqrt{a+x} = a - x^2$$

Next, I could square both sides again, treating the equation: ...

...

**Reasoning capabilities of LLMs can be incentivized purely through RL with outcome based, verifiable rewards.
No process rewards or human step-level annotations necessary.**

You can try RLVR and GRPO yourself



Wait, this seems complicated. Let's list the coordinates in order and apply the shoelace formula carefully.

List of vertices in order: $A(u, v)$, $B(v, u)$, $C(-v, u)$, $D(-v, -u)$, $E(v, -u)$.

Case 1: $u = 11$

$$11(11 + 3v) = 45111 + 3v = \frac{451}{11} = 413v = 40v = \frac{40}{3}$$

But v must be an integer, so $u = 11$ is invalid.

Case 2: $u = 41$

$$41(41 + 3v) = 45141 + 3v = \frac{451}{41} = 113v = 0v = 0$$

But v must be positive, so $u = 41$ is invalid.

Wait, there's a mistake here. Let's re-examine the factors.

In Practice, RL Experiments can be challenging and costly

- 560 hyper-parameters in verl (a popular LLM RL framework)
- Complicated: 50K+ lines of code; multi-GPU training
- Computational cost; memory requirements
- Rewards, Verifiers, Tools, RL environments
- **Training stability**

Conclusions

“With the advent of pure RL methods such as DeepSeek-R1, the future holds immense potential for **solving any task** that can be effectively evaluated by a **verifier, regardless of its complexity for humans**. Machines equipped with such advanced RL techniques are poised to **surpass human capabilities** in these domains, driven by their ability to optimize performance iteratively through trial and error.

Ethics and Societal Impact

- LLM RL experiments are expensive
 - Only a few companies can afford them
- Concentration of intellectual output and wealth to a few companies
- What will future AGI-capable models do to us?

Thank you!

Questions?